

CAP693 – Programming in C

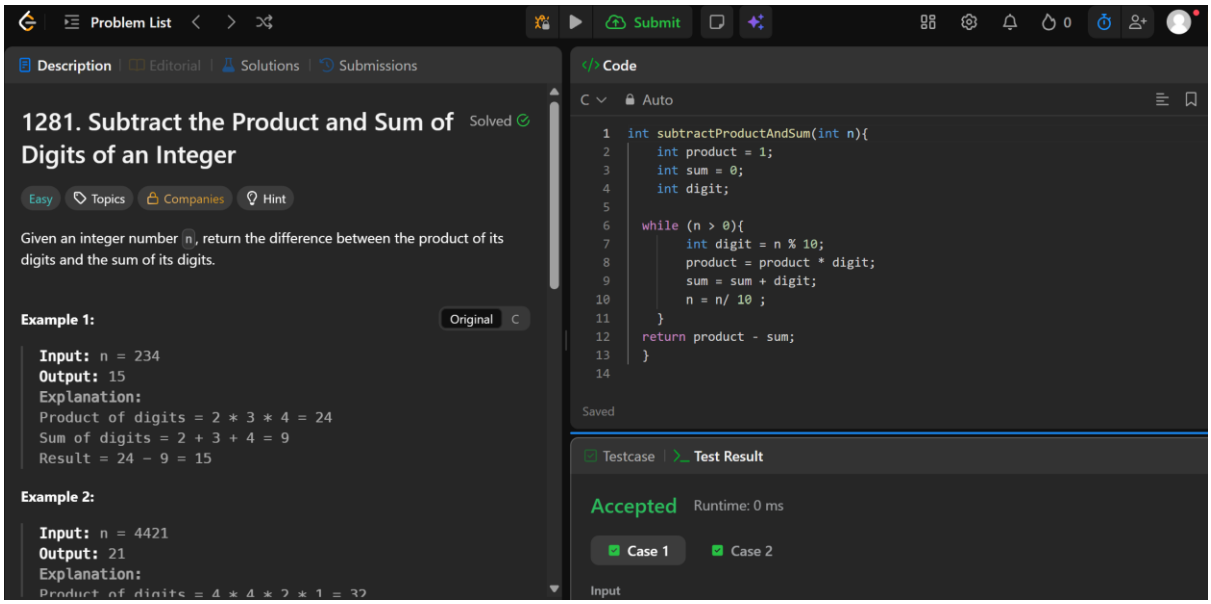
CONTINUOUS ASSESSMENT 3 (CA3)

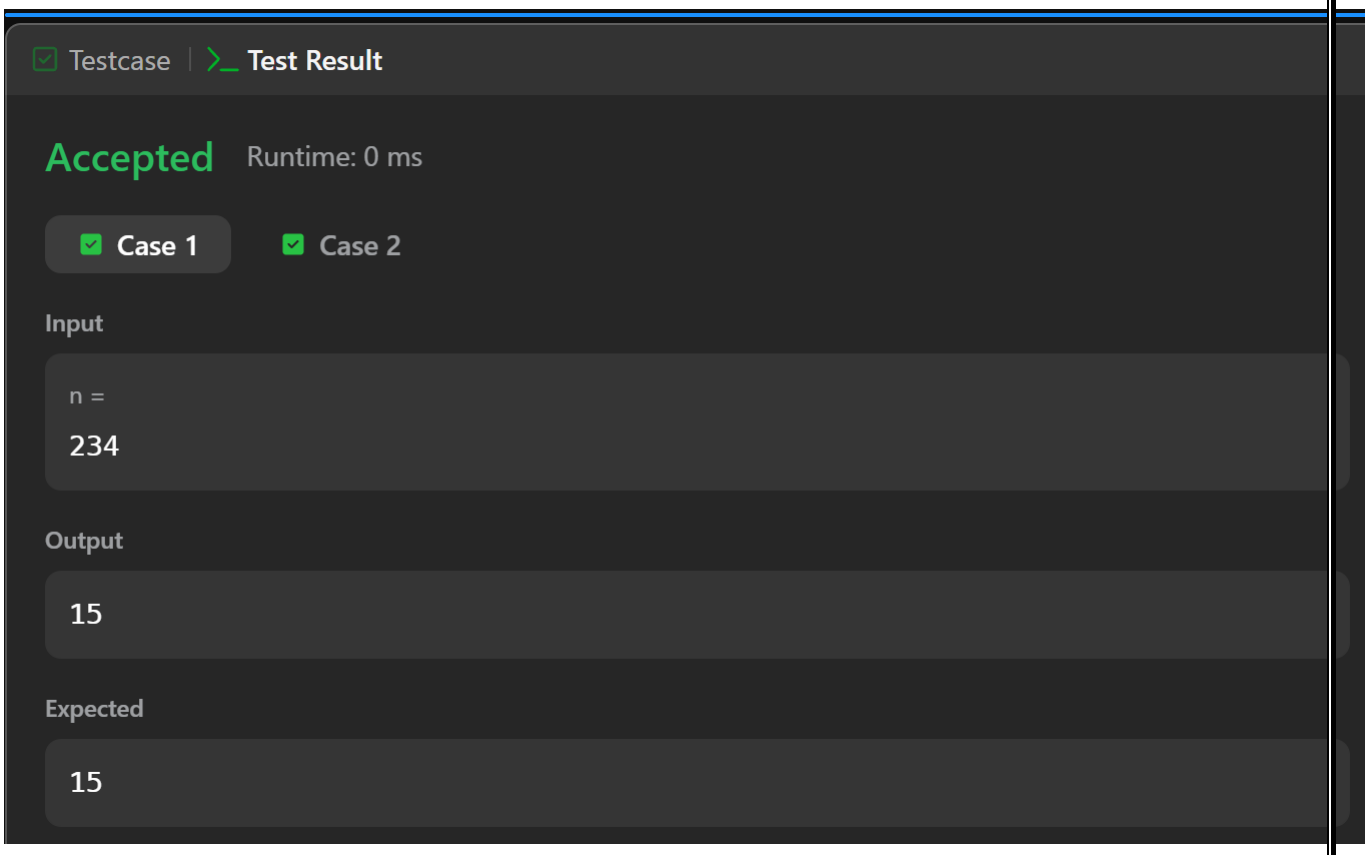
LeetCode Problem Solving using C and GitHub Submission

NAME: AFRIN JAMAL	***COURSE CODE: CAP693***
ID NO: 12630229	***SUBJECT: PROGRAMMING IN C***
SECTION: DIP2648	***SUBMISSION DATE: 13/11/2026***

Q1. Leet code problem no. 1281

Subtract the product and sum of digits of an integer
Loops, arithmetic operators, expressions

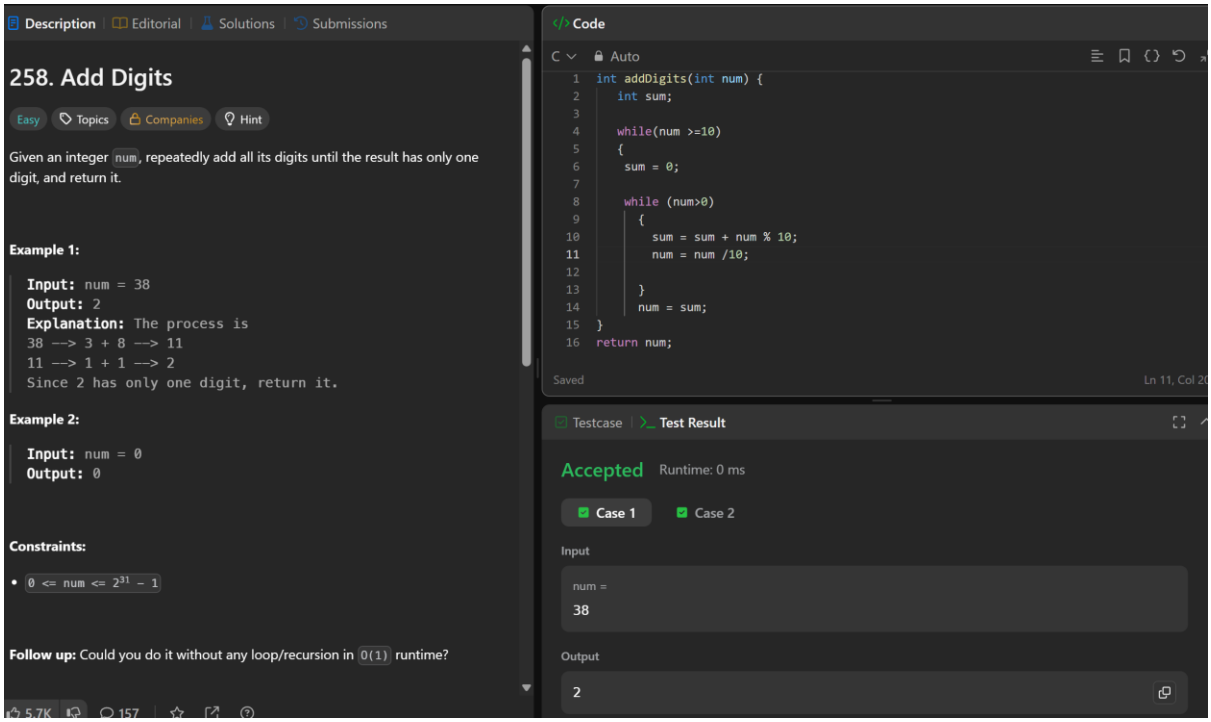




Q2. Leet code Problem no. 258

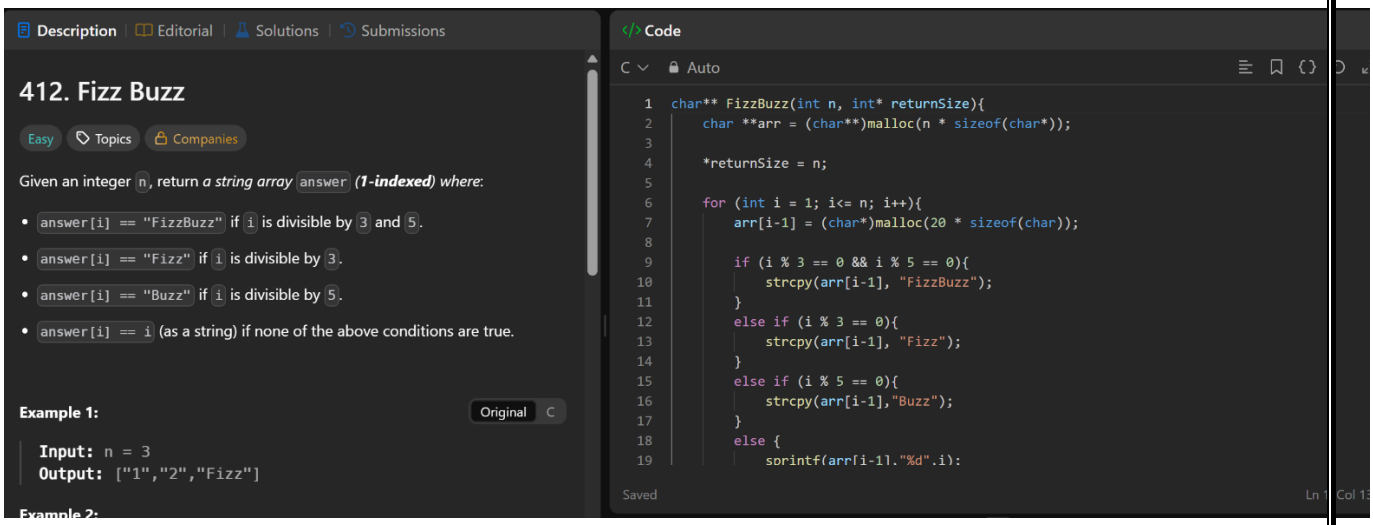
Add Digits
Loops, arithmetic, I/O

Solution:



Q3. Leet Code problem no. 412

FizzBuzz
if else, loops, modulus operator
Solution:



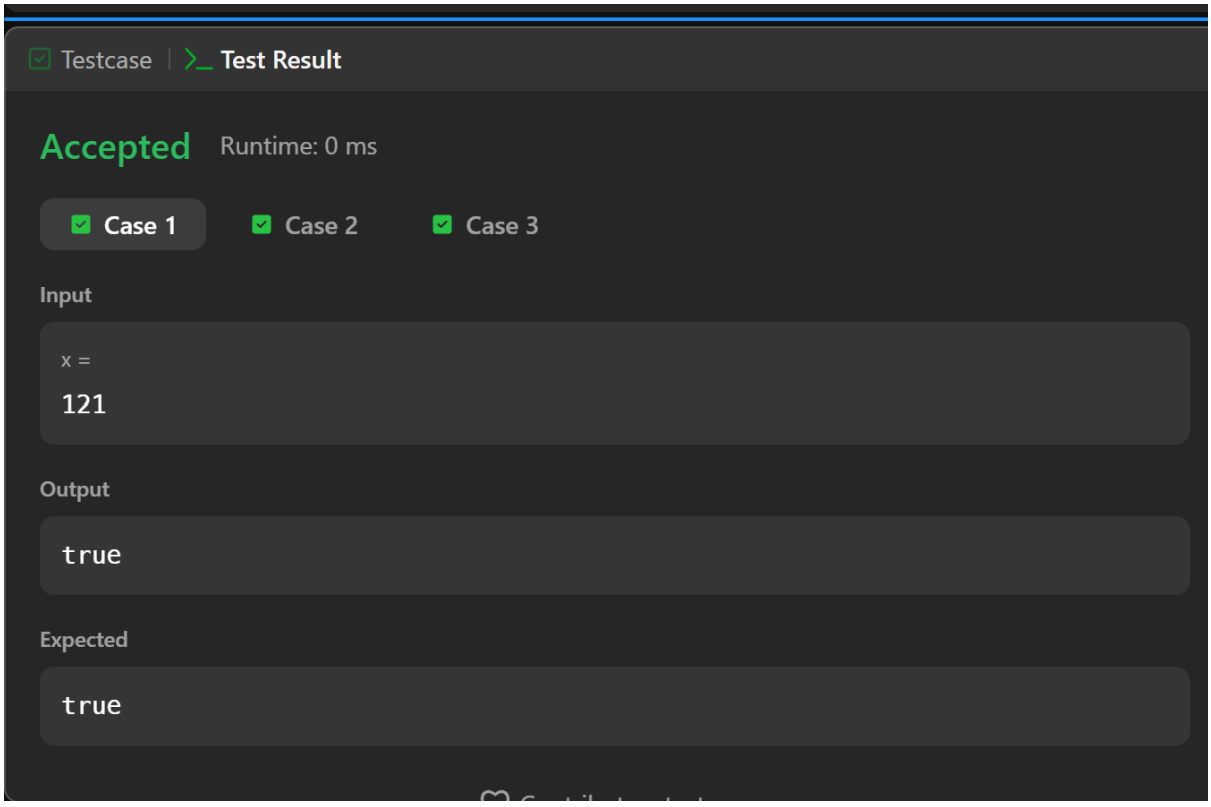
Q4. Leet code problem no. 9
Palindrome Numbers
Loops, relational and logical operators

Code

C Auto

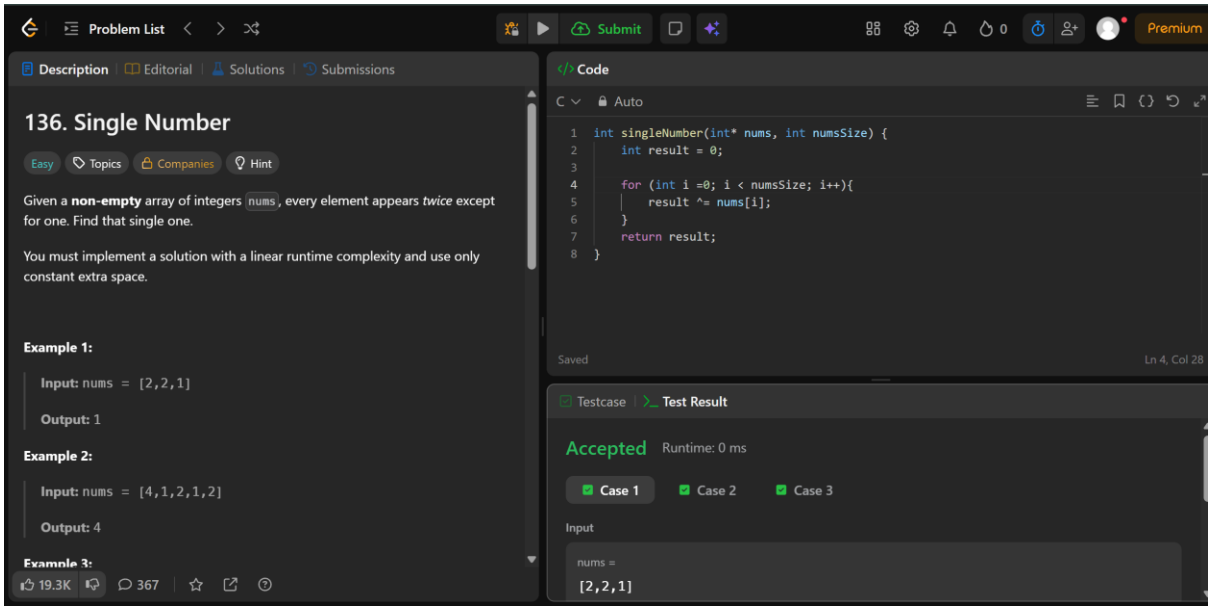
```c
bool isPalindrome(int x) {
    if (x < 0) {
        return false;
    }
    long long original = x;
    long long reversed = 0;

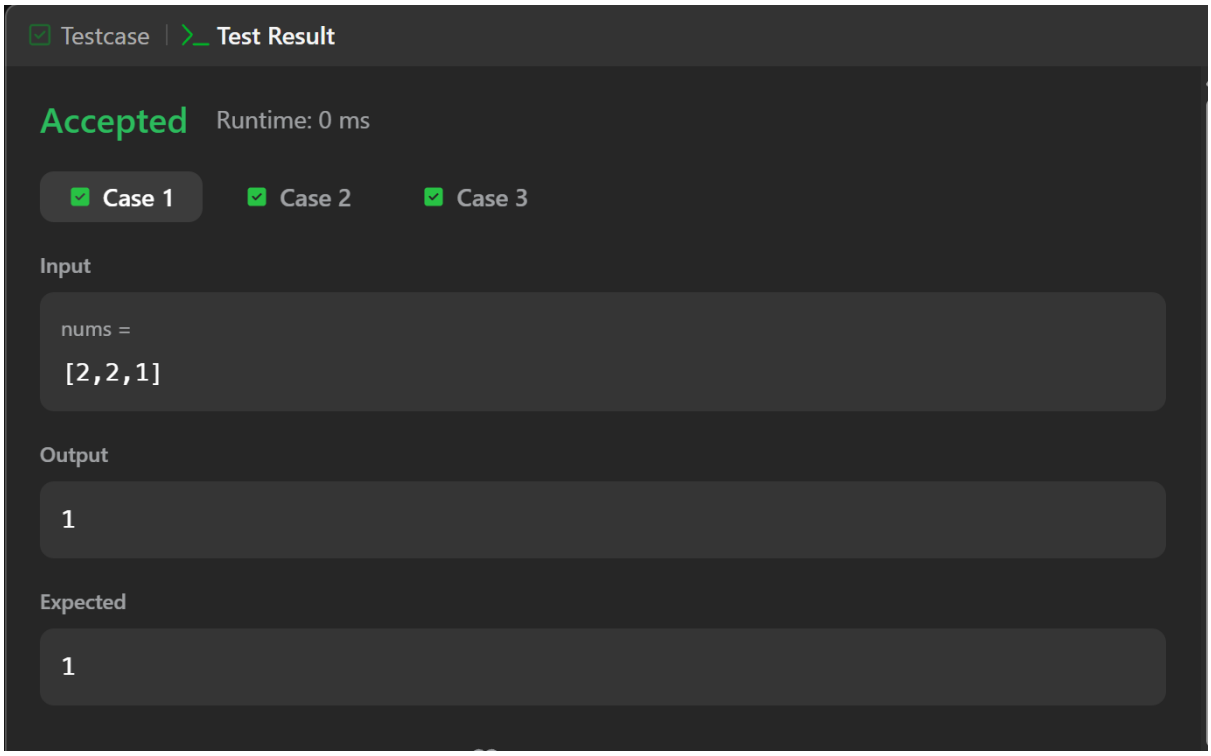
    while (x >0){
        int remainder = x % 10;
        reversed = reversed * 10 + remainder;
        x /= 10;
    }
    return original == reversed;
}
```



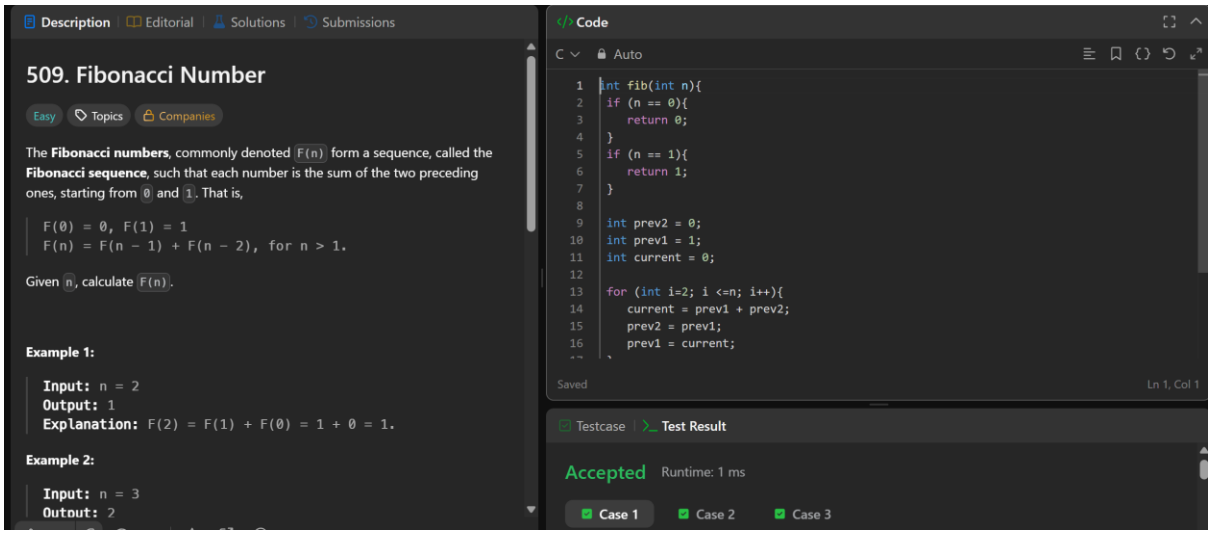
Q5. Leet code problem no. 136

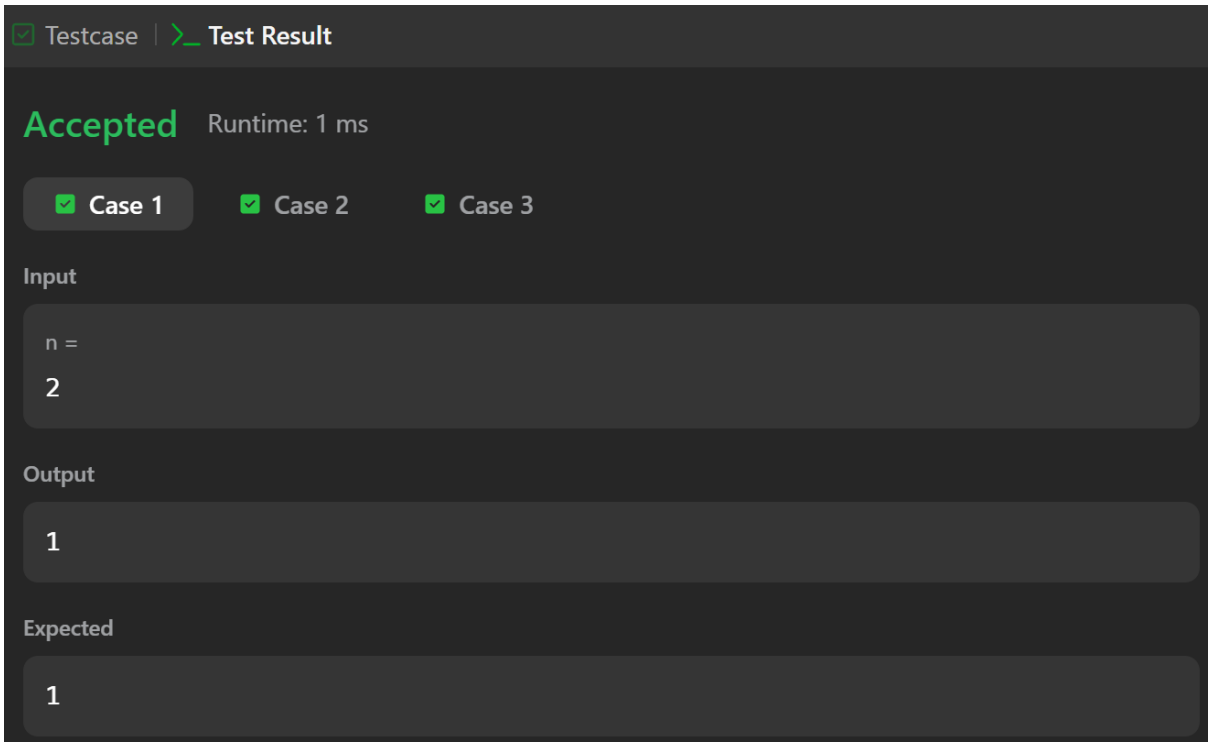
Single number
Bitwise, XOR, loops





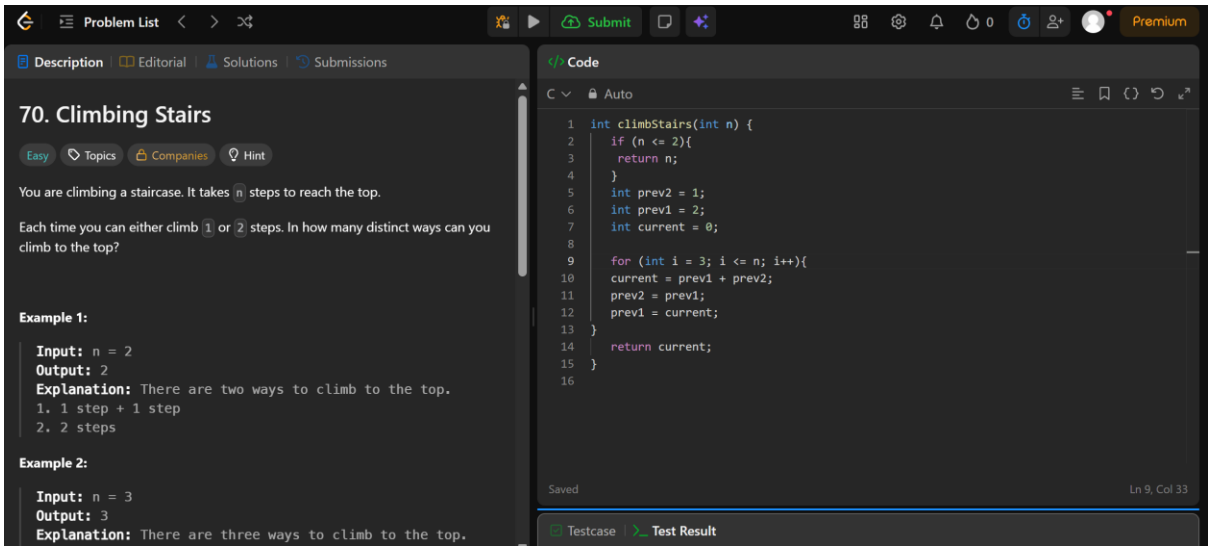
Q6. Leet code problem no. 509
Fibonacci Number
Recursion, function definition





Q7. Leet code problem no. 70

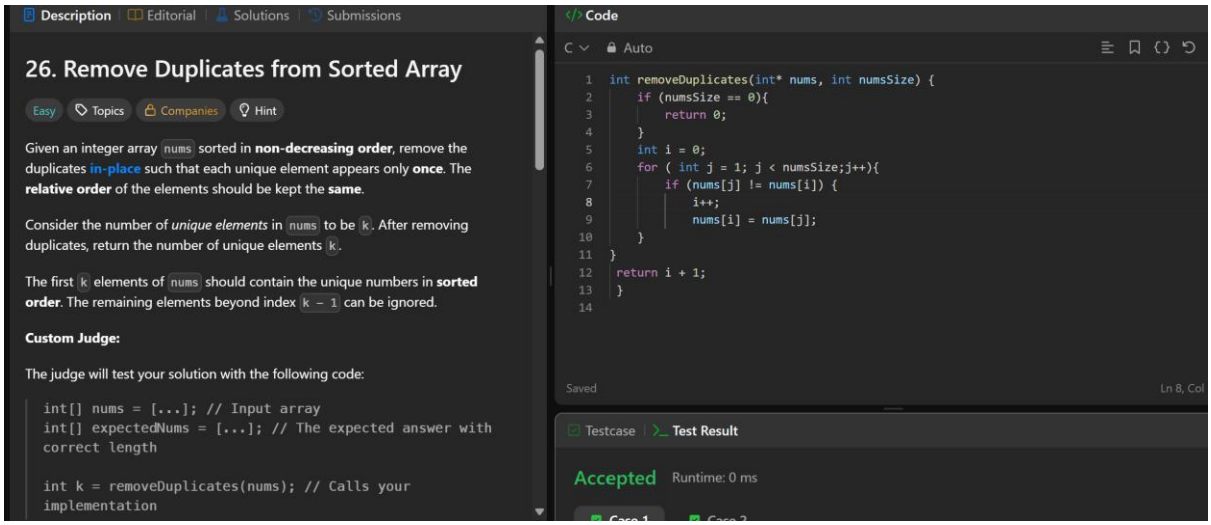
Climbing Stairs
Recursion, functions

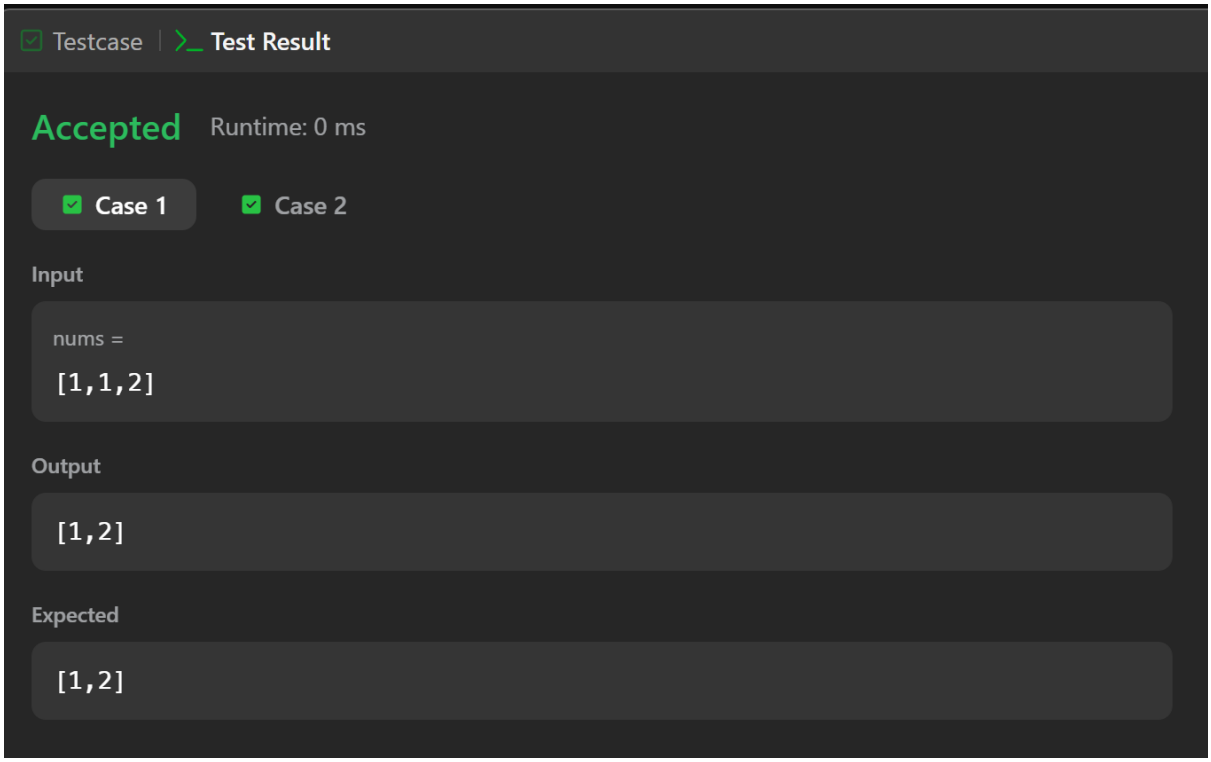




Q8. Leet code problem no. 26
Remove duplicates from sorted arrays

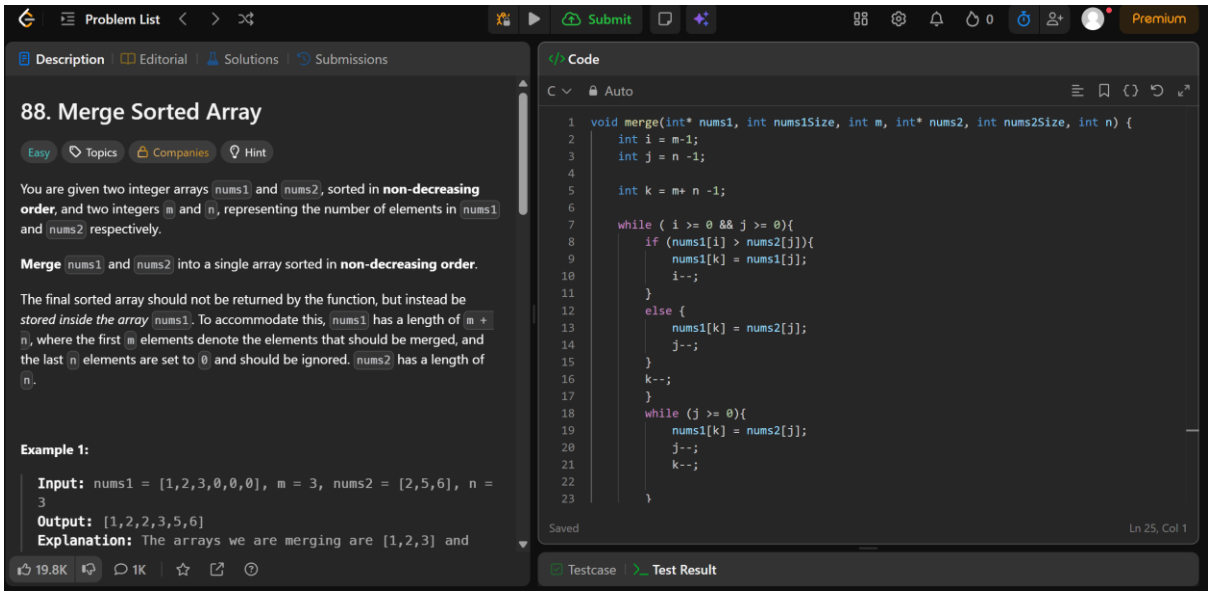
1D arrays, traversal, array modifications

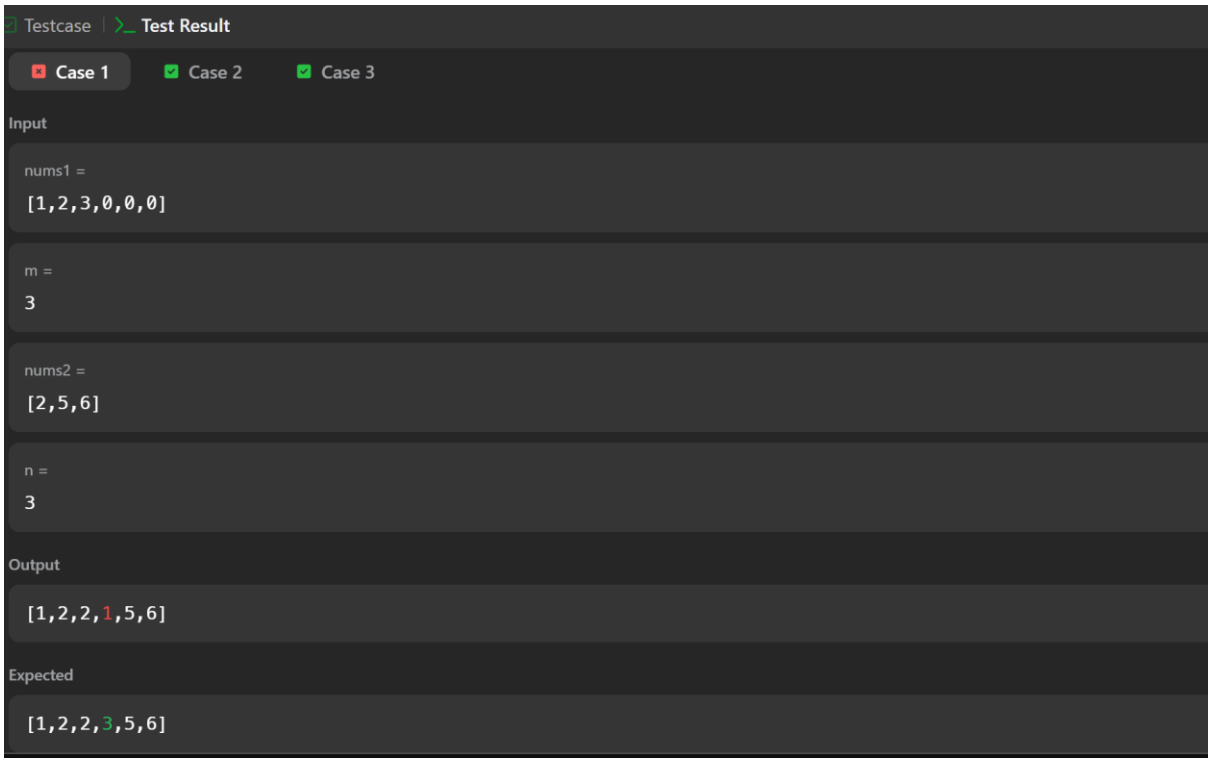


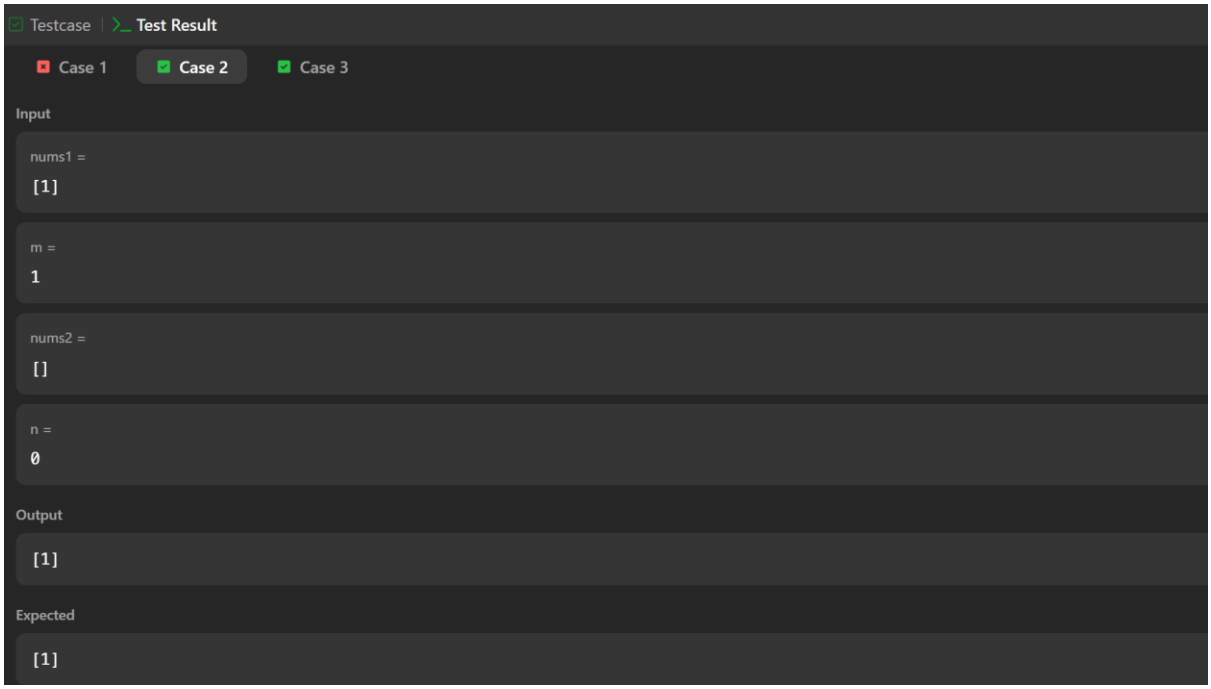


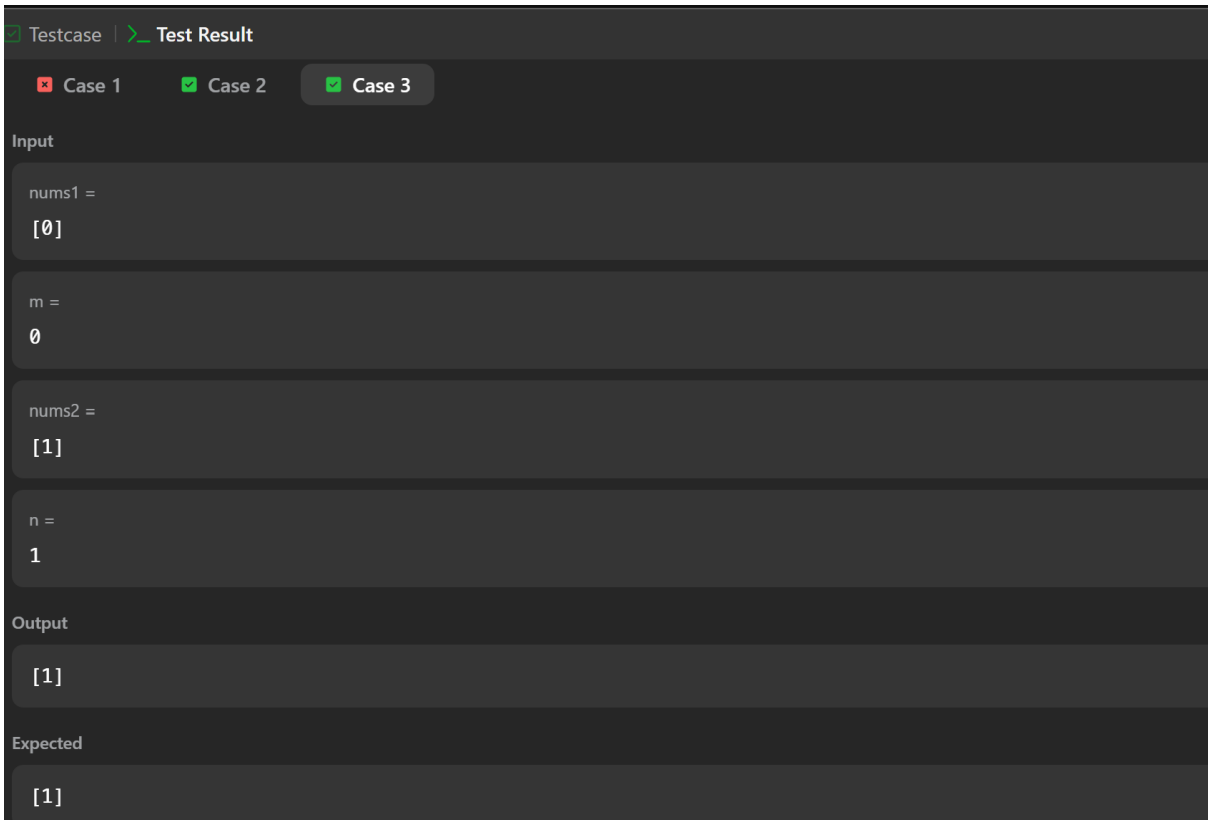
Q9. Leet code Problem no. 88
Merge sorted array

1D arrays, merging, passing arrays to functions



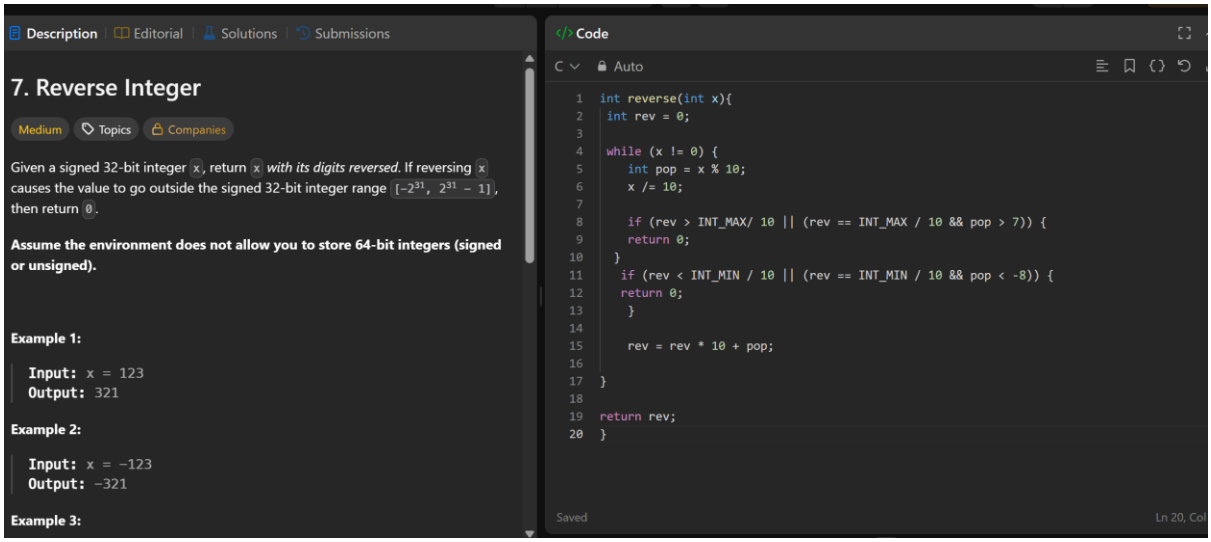


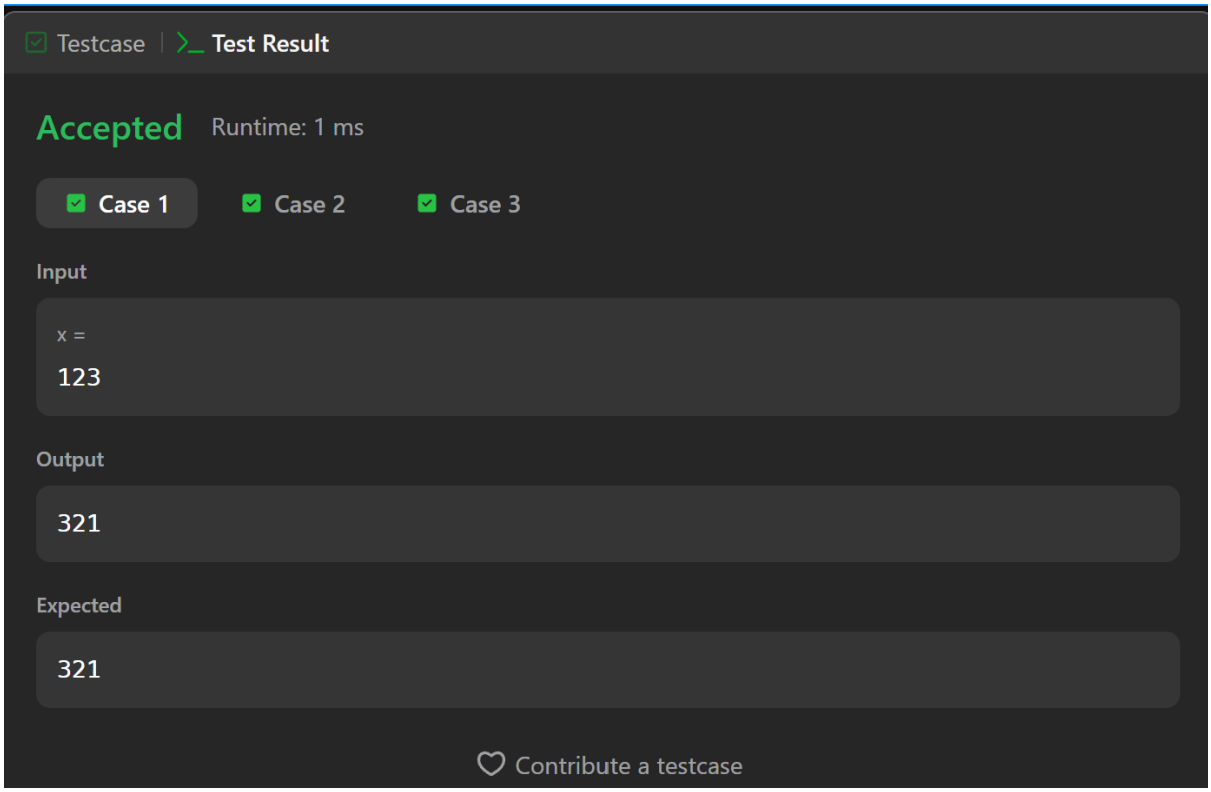

Testcase
Test Result
Case 1
Case 2
Case 3
Input
nums1 =
[1]
m =
1
nums2 =
[]
n =
0
Output
[1]
Expected
[1]

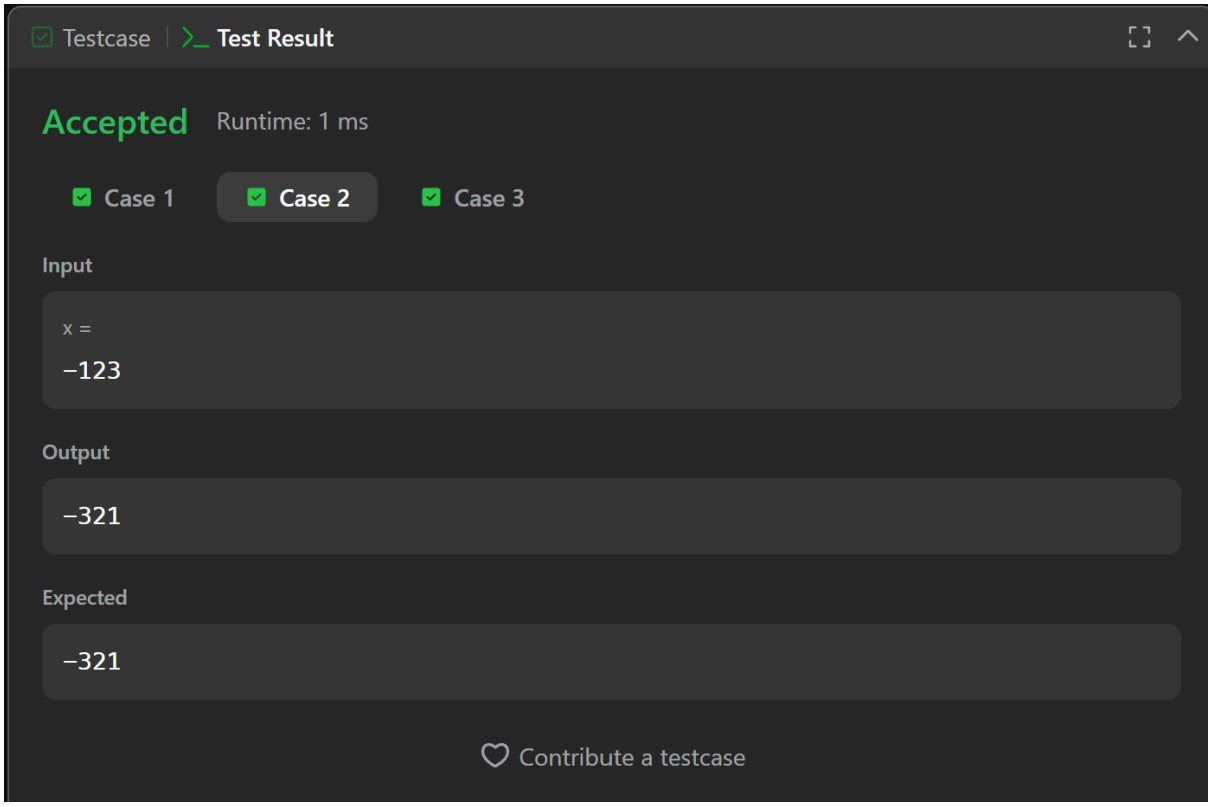

Testcase
Test Result
Case 1
Case 2
Case 3
Input
nums1 =
[0]
m =
0
nums2 =
[1]
n =
1
Output
[1]
Expected
[1]

Q10. Leet code problem no. 7
Medium Reverse Integer

Loops, type conversion, type modifiers, overflow






Testcase
Test Result
Accepted
Runtime: 1 ms
Case 1
Case 2
Case 3
Input
x =
−123
Output
−321
Expected
−321
Contribute a testcase

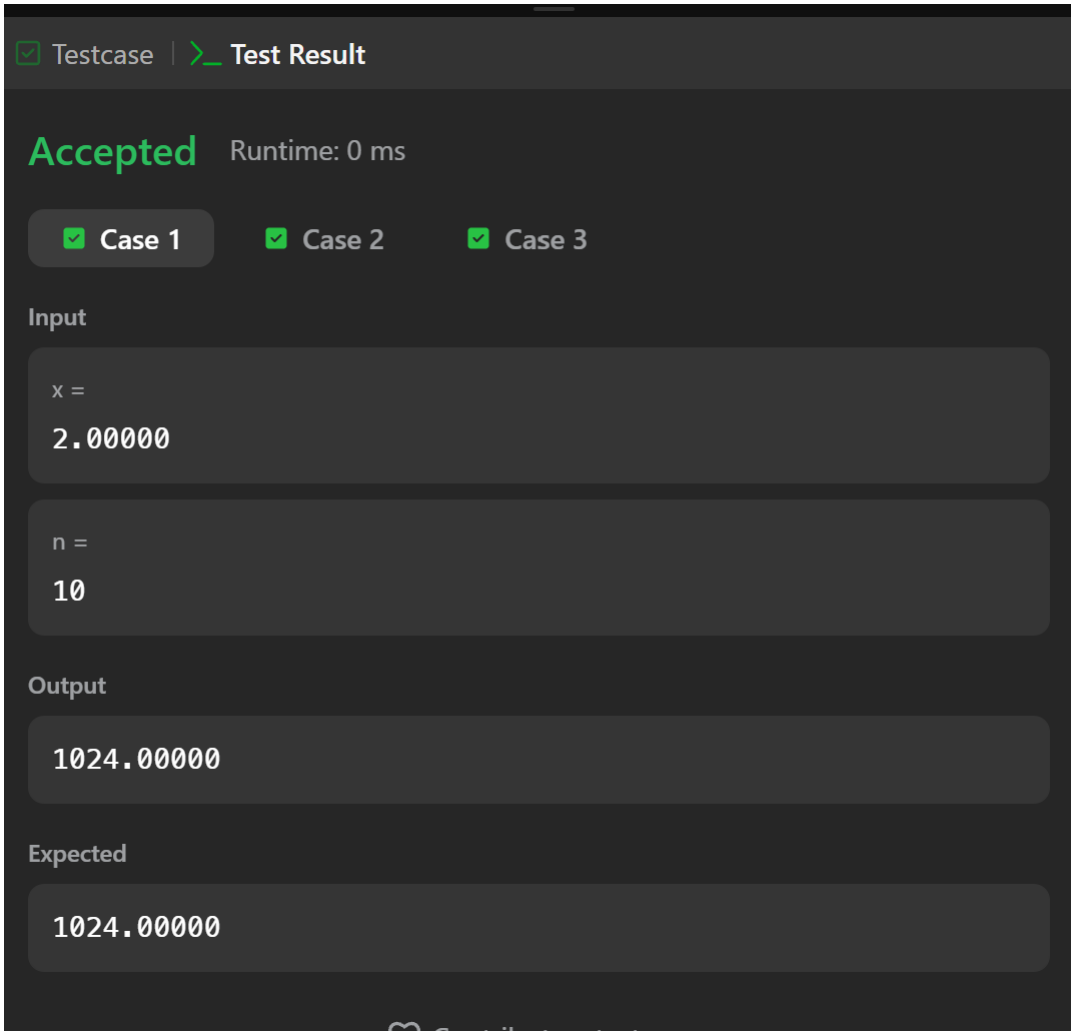
Q11. Leet code problem no. 50
Medium Power (x,n) Recursion, function

Code

C Auto

```c
double myPow(double x, int n) {
  long long binform = n;
  if (binform < 0){
    x = 1 / x;
    binform = - binform;
  }

  double ans = 1.0;
  while (binform > 0){
      if (binform % 2 == 1) {
       ans *= x;
    }
    x *= x;
    binform /= 2;
  }
  return ans;
}
```


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
x =
2.00000
n =
10
Output
1024.00000
Expected
1024.00000

Q12. LeetCode problem no. 22

Medium Generate Parenthesis
Recursion, arguments, passing, character arrays

Code

C Auto

```c
void backtrack(char** result, int* returnSize, char* current, int open, int close, int n) {
    if (strlen(current) == 2 * n){
        result[*returnSize] = strdup(current);
        (*returnSize)++;
        return;
    }
    if (open < n) {
        int len = strlen(current);
        current[len] = '(' ;
        current[len + 1] = '\0' ;
        backtrack(result, returnSize, current, open + 1, close, n);
        current[len]= '\0' ;
    }

    if (close < open){
        int len = strlen(current);
        current[len] = ')' ;
        current[len + 1] = '\0' ;
        backtrack(result, returnSize, current, open, close + 1, n);
        current[len] = '\0' ;
    }
}
char** generateParenthesis(int n, int* returnSize) {
    *returnSize = 0;
    char** result = (char**)malloc(15000 * sizeof(char*));
    char* current = (char*)malloc((2*n + 2) * sizeof(char));
    current[0] = '\0' ;

    backtrack(result, returnSize, current, 0, 0, n);

    free(current);
    return result;
}
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n =
3

Output

["((()))","(()())","(())()","()(())","()()()"]

Expected

["((()))","(()())","(())()","()(())","()()()"]

Q13. Leet Code problem no. 189
Medium Rotate Array
1D arrays, traversal, array manipulation

Code

C Auto

```c
void rotate(int* nums, int numsSize, int k) {
    k = k % numsSize;

    int temp [numsSize];

    for (int i = 0; i < numsSize; i++){
        temp[(i + k) % numsSize] = nums[i];
    }
    for (int i = 0; i < numsSize; i++)  {
        nums[i] = temp[i];
    }
}
```

Saved Ln 10, Col 27

Code

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,2,3,4,5,6,7]

k =
3

Output

[5,6,7,1,2,3,4]

Expected

[5,6,7,1,2,3,4]

Q14. Leet Code problem no. 48

Medium Rotate Image
2D arrays, matrix, processing

Description | Editorial | Solutions | Submissions

48. Rotate Image

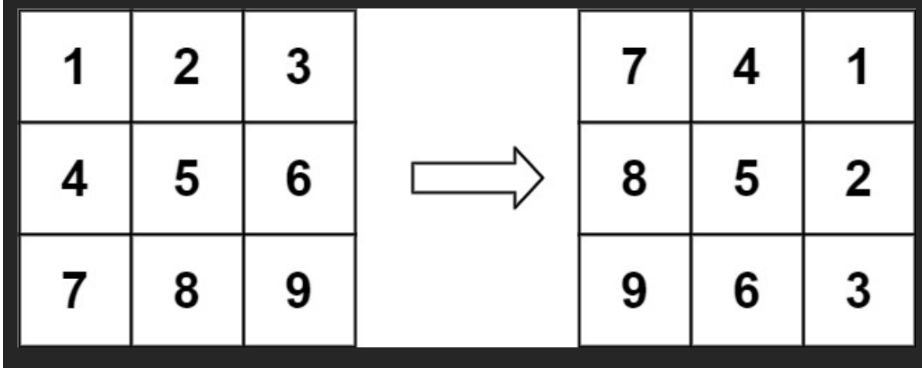
Attempted

Medium Topics Companies

You are given an `n x n` 2D `matrix` representing an image, rotate the image by **90** degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:



Code

C Auto

```c
void rotate(int** matrix, int matrixSize, int* matrixColSize) {
    for (int i = 0; i < matrixSize; i++){
     for (int j = i + 1; j < matrixSize; j++) {
         int temp = matrix[i][j];
         matrix[i][j] = matrix[j][i];
         matrix[j][i] = temp;
     }
}
  for (int i = 0; i < matrixSize; i++);{
     for (int j = 0; j < matrixSize / 2; j++){
         int temp = matrix[1][j];
         matrix[1][j] = matrix[1][matrixSize - 1 - j];
         matrix[1][matrixSize - 1 - j] = temp;
     }
  }
}
```

Wrong Answer Runtime: 0 ms Diff

Case 1 Case 2

Input

matrix =

```
[[1,2,3],[4,5,6],[7,8,9]]
```

Output

```
[[1,4,7],[8,5,2],[3,6,9]]
```

Expected

```
[[7,4,1],[8,5,2],[9,6,3]]
```

Q.15 Leet code problem no. 51

Hard: N- Queens
Recursion, 2D arrays, strings

Code

C Auto

```c
int isSafe(int row, int col, char** board, int n){
    for (int i = 0; i < row; i++){
        if (board[i][col] == 'Q') return 0;
    }
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--){
        if (board[i][j] == 'Q') return 0;
    }
    for (int i = row - 1, j = col + 1; i >= 0 && j <n; i--, j++){
        if (board[i][j] == 'Q') return 0;
    }
    return 1;
    }
void solve(int row, int n, char** board, char** result, int* returnSize, int* returnColumnSizes){
    if (row == n){
    char* cur = (char*)malloc(n* n * sizeof(char));
    char** singleResult = (char**)malloc(n * sizeof(char*));
    for (int i = 0; i < n; i++){
        singleResult[i] = cur + i *n;
        memcpy(singleResult[i], board[i], n);
        singleResult[i][n] = '\0' ;
    }
    int idx = (*returnSize)++;
    *result = (char*) realloc(*result, (*returnSize) * sizeof(char*)) ;
    (*result)[idx] =(char*) singleResult;
```

Code

C Auto

```c
    for (int col = 0; col < n; col++){
        if (isSafe(row, col, board, n)){
            board[row][col] = 'Q';
            solve(row + 1, n, board, result, returnSize, returncolumnSizes);
            board[row][col] = '.';
        }
    }
}

char*** solveNQueens(int n, int* returnSize, int** returnColumnSizes) {
    *returnSize = 0;
    *retrunColumnSizes = NULL;
    char*** result = NULL;

    char** board = (char**) malloc( n * sizeof(char*));
    for (int i = 0; i < n; i++){
        board[i] = (char*)malloc((n + 1) * sizeof(char));
        for (int j = 0; j < n; j++){
            board[i][j] = '.' ;
        }
        board[i][n] = '\0' ;
        }
        solve (0, n, board, &result, returnSize, returncolumnSizes);

        for (int i = 0; i < n; i++){
```

Code

C Auto

```c
    char*** result = NULL;

    char** board = (char**) malloc( n * sizeof(char*));
    for (int i = 0; i < n; i++){
        board[i] = (char*)malloc((n + 1) * sizeof(char));
        for (int j = 0; j < n; j++){
            board[i][j] = '.' ;
        }
        board[i][n] = '\0' ;
        }
        solve (0, n, board, &result, returnSize, returncolumnSizes);

        for (int i = 0; i < n; i++){
            free(board[i]);
        }
        free(board);

        return result;
    }

}
```

Testcase | Test Result

Compile Error

```
solution.c: In function ‘solve’
Line 24: Char 20: error: assignment to ‘char’ from ‘char *’ makes integer from pointer wi
thout a cast [-Wint-conversion] [solution.c]
   24 |     (*result)[idx] =(char*) singleResult;
      |                    ^
```

Q16. Leet code problem no. 42

Trapping rain water
1D arrays, traversal, loops

Description | Editorial | Solutions | Submissions

42. Trapping Rain Water

Hard | Topics | Companies

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Code

C | Auto

```c
int trap(int* height, int heightSize) {
  if (heightSize <=2) return 0;

  int left = 0, right = heightSize - 1;
  int left_max = 0, right_max = 0;
  int water_trapped = 0;

 while (left < right) {

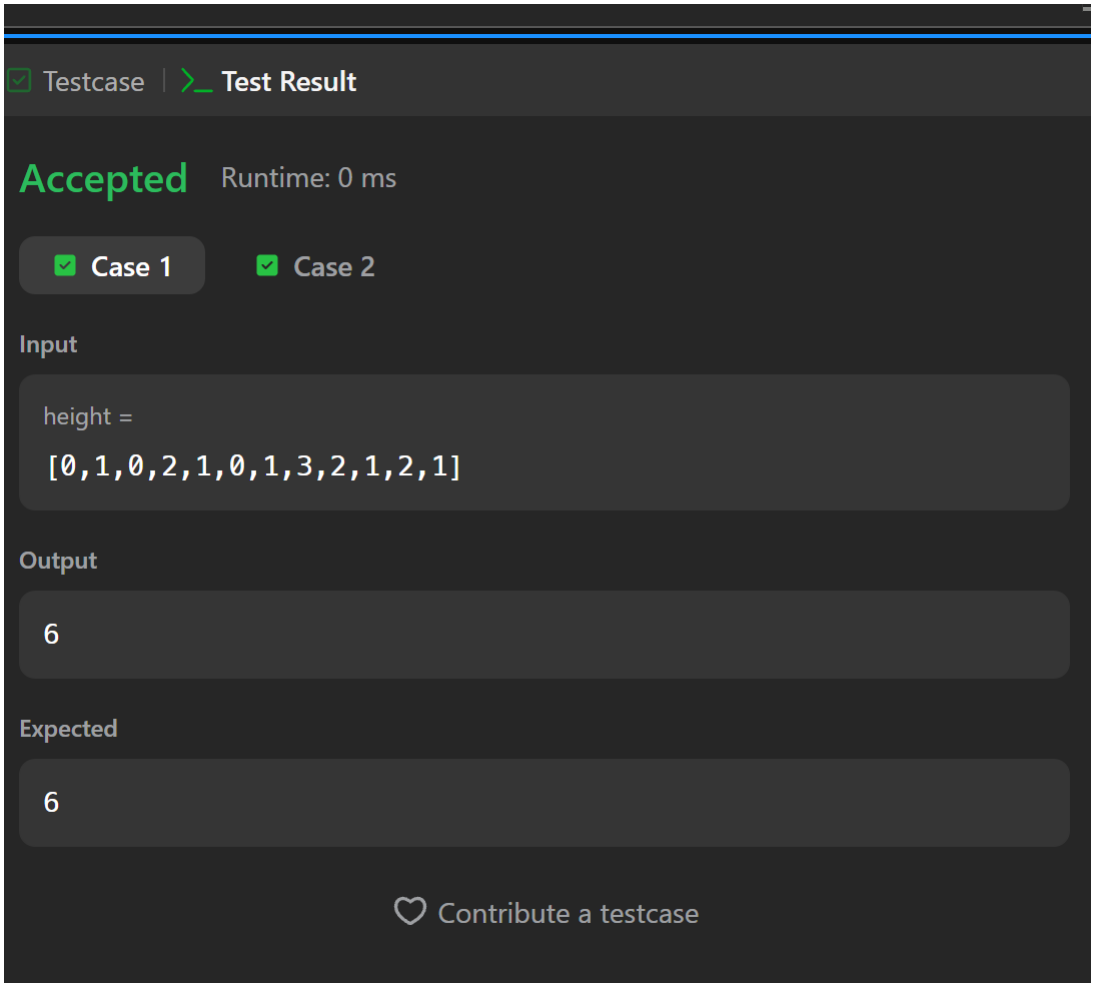
    if (height[left] < height[right]){
        if (height[left] >= left_max){
            left_max = height[left];

        }  else{

        water_trapped += left_max - height[left];
  }
  left++;
} else {
    if (height[right] >= right_max){
        right_max = height[right];
    }else{
        water_trapped += right_max - height[right];

    }
```

Saved


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
height =
[0,1,0,2,1,0,1,3,2,1,2,1]
Output
6
Expected
6
Contribute a testcase